

Aave V4: A Modular Liquidity Protocol with Risk-Adjusted Borrowing Costs

Emilio Frangella
emilio@aave.com

Stani Kulechov
stani@aave.com

Miguel Martinez
miguel@aave.com

Dhairya Sethi
dhairya@aave.com

Yan Bin Man
yan@aave.com

Alexandru-Vlad Niculae
alex.n@aave.com

Valentin Viger
valentin@aave.com

August 26, 2026

Abstract

This document describes Aave V4, a permissionless, automated and non-custodial liquidity protocol originally built for the Ethereum Virtual Machine. Aave V4 replaces the monolithic architecture of previous iterations with a modular and extensible design that increases capital efficiency while providing better risk segregation and more granular control. The protocol prices borrowing costs as a function of collateral quality, versions its risk parameters so that configuration changes no longer disturb existing positions, and restores borrowers to a target health factor rather than liquidating a fixed fraction of their debt.

1 Introduction

Lending protocols have become the foundational infrastructure in decentralized finance (DeFi), allowing permissionless borrowing and lending of digital assets. Since its inception, Aave has iterated through multiple protocol generations. Aave V2 introduced stable-rate borrowing and flash loans; Aave V3 added Portals, eModes, and Isolation mode. Aave V3 has had multiple iterations over the last three years and represents the foundational component of the Aave ecosystem. Aave V3 has proven efficiency and scalable capabilities, but multiple areas of improvement have been identified and addressed with Aave V4 through an architecture redesign and a set of new features:

1. A **Hub-and-Spoke architecture** separating liquidity coordination from borrowing logic.
2. A **Risk Premium system** pricing borrowing costs as a function of collateral asset class.
3. A **Dynamic Risk Configuration** to improve risk configuration granularity.
4. A **redesigned Liquidation Engine** to improve capital efficiency and reduce the risk of excessive liquidations.
5. A **Reinvestment Module** for deploying idle liquidity to external risk-off strategies.

2 Architecture

2.1 Hub-and-Spoke Model

At the highest level of abstraction, the protocol is organized into two layers. The **Hub** serves as the central liquidity coordinator, manages asset registration, interest accrual, share and Risk Premium accounting, fee collection, deficit tracking, float liquidity reinvestment and per-Spoke liquidity caps. Multiple **Spokes** connect to the Hub and implement user-facing operations, with the standard Spoke implementation providing supply, withdraw, borrow, repay, and liquidation.

Each Spoke may define its own oracle configuration, risk parameters, collateral policies, and liquidation settings for the same underlying assets on the same Hub. The Hub enforces per-Spoke *add caps* C_a and *draw caps* C_d , expressed in whole asset units as `uint40`, which bound each Spoke's contribution to and extraction from the shared liquidity pool.

This separation yields three key properties:

1. **No liquidity migration**: any new borrowing strategy is added as a Spoke connected to existing Hub liquidity, allowing deposits to remain in place regardless of its collateral model.
2. **Risk isolation**: risk stays isolated, preventing a misconfigured or exploited Spoke from draining the underlying liquidity.
3. **Flexible collateral design**: each Spoke owns its user-facing logic, enabling non ERC-20 collateral models without requiring changes to the Hub.

2.2 Core Hub Operations

The Hub exposes four atomic liquidity primitives:

1. **add(asset, amount)**: deposits liquidity, minting *added shares* for the calling Spoke.
2. **remove(asset, shares)**: redeems added shares for underlying tokens.
3. **draw(asset, amount)**: borrows from the Hub, minting *drawn shares* against the Spoke.
4. **restore(asset, amount, premiumAmount)**: repays drawn debt and premium amount, burning drawn

shares.

Every state-mutating Hub function calls `accrue()` first, which performs basic state update operations, including updating the drawn index via linear interest and accruing unrealized fees.

2.3 Share Accounting

2.3.1 Added Shares (Supply)

Added shares use a virtual-shares pattern to prevent donation attacks:

$$\text{shares} = \left\lfloor \frac{\text{assets} \cdot (S + V)}{A + V} \right\rfloor \quad (1)$$

where S is total added shares, A is total added assets, and $V = 10^6$ is the virtual offset applied to both numerator and denominator. The total added assets backing shares are:

$$A = L + W + \left\lceil \frac{D \cdot I_d + P_{\text{RAY}} + \Delta_{\text{RAY}}}{10^{27}} \right\rceil - F_r - F_u \quad (2)$$

where L is liquidity, W is swept (reinvested) amount, D is drawn shares, P_{RAY} is premium debt, Δ_{RAY} is deficit, and F_r, F_u are realized and unrealized fees, respectively.

2.3.2 Drawn Shares (Debt)

Drawn shares track borrower's debt using a monotonically increasing *drawn index* I_d in RAY precision (10^{27}). Conversions follow:

$$\text{assets} = \text{shares} \cdot I_d / 10^{27}, \quad \text{shares} = \text{assets} \cdot 10^{27} / I_d \quad (3)$$

Rounding is always *against the borrower*: drawn debt is rounded up when minted and down when repaid, and shares are rounded up when minted.

The index accrues linearly:

$$I_d(t) = \left\lceil I_d(t-1) \cdot \left(\text{RAY} + \frac{r_d \cdot \Delta t}{Y} \right) \right\rceil_{27} \quad (4)$$

where r_d is the current drawn rate, $\Delta t = t - t_{\text{last}}$ in seconds, $Y = 31,536,000$ (seconds per year), and $\lceil \cdot \rceil_{27}$ denotes RAY-precision ceiling multiplication.

3 Risk Premiums

3.1 Motivation

In Aave V3, all borrowers of a given asset pay the same utilization-dependent rate. For example, a borrower posting collateral with technical or third-party risk pays identical rates as one posting ETH, yet the former poses greater challenges for the protocol. This systematic mispricing subsidizes risky borrowers at the expense of depositors, or negatively affects organic borrowers with lower risk collateral.

3.2 Collateral-Weighted Risk Premium

Aave V4 assigns to each asset a *Collateral Risk* value $r_i \in [0, 100,000]$ basis points per pair. For a

borrower with total debt value D and collateral positions $\{(v_1, r_1), \dots, (v_n, r_n)\}$ sorted by ascending risk, the User Risk Premium ρ is computed via a greedy covering algorithm:

$$\rho = \frac{\sum_{i=1}^n r_i \cdot \min(v_i, D_{\text{rem},i})}{\sum_{i=1}^n \min(v_i, D_{\text{rem},i})} \quad (5)$$

where $D_{\text{rem},i}$ is the uncovered debt after applying collaterals $1, \dots, i-1$. Ties in risk are broken by value (descending). The result is stored in basis points as `uint24`. The total borrower rate for a user U becomes:

$$r_{\text{total}} = r_{\text{drawn}}(U) + \rho \quad (6)$$

3.3 Premium Debt Accounting

Premium debt accrues as a separate component alongside drawn debt. The protocol maintains per-position *premium shares* P_s and a signed *premium offset* O_p (RAY-scaled). The premium debt is:

$$P_{\text{debt}} = \lceil P_s \cdot I_d - O_p \rceil \quad (7)$$

At borrow time, premium shares are set to $P_s = \lceil D_s \cdot \rho / 10,000 \rceil$ (where D_s are drawn shares) and the offset is calibrated so that $P_{\text{debt}} = 0$: $O_p = P_s \cdot I_d$. Over time, as I_d grows, premium debt accrues naturally:

$$P_{\text{debt}}(t) = P_s \cdot (I_d(t) - I_d(t_0)) \quad (8)$$

When a user's collateral composition changes or ρ is recalibrated, the offset is adjusted so that accumulated premium debt is preserved while share counts reflect the new risk premium and the premium debt continues to accrue according to the refreshed ρ . During repayment, **premium debt is repaid first**; only after premium is fully cleared does repayment reduce drawn debt.

4 Dynamic Risk Configuration

4.1 Versioned Parameters

Aave V4 maintains multiple *Dynamic Reserve Configurations* per Reserve simultaneously, each identified by a `uint32` key. Each configuration stores three parameters: *Collateral Factor* (BPS, $< 10,000$), *Max Liquidation Bonus* (BPS, $\geq 10,000$), and *Liquidation Fee* (BPS, $\leq 10,000$). The Dynamic Risk Configuration allows the implementation of mechanisms like the LTV 0 present in V3 without additional logic, therefore LTV has been deprecated and Liquidation Threshold has been replaced by Collateral Factor.

4.2 Snapshot-and-Rebind

When governance creates a new configuration, existing user positions retain their snapshot key. A position is rebound to the latest configuration **only** when the user takes a risk-increasing action (borrowing, withdrawing collateral, or disabling a collateral flag) *and* the position

remains healthy under the new parameter set. This solves one of the major challenges of Aave V3 where changing the liquidation threshold affects both existing and new users at the same time, leading to suboptimal risk configuration. Governance can still change older configurations to allow for protocol wide changes when needed (for example, offboarding a collateral).

5 Liquidation Engine

5.1 Health Factor

For a user with collateral positions $\{(V_i, CF_i)\}$ and total debt value D :

$$HF = \frac{\sum_i CF_i \cdot V_i \cdot 10^{14}}{D} \quad (9)$$

where V_i is collateral value (amount * price returned by the Oracle) and CF_i is the collateral factor in BPS (the additional 10^{14} term is included to match WAD decimals). A position is liquidatable when $HF < 10^{18}$ (1.0 in WAD). If $D = 0$, $HF = \infty$.

5.2 Target Health Factor Restoration

Rather than having a fixed close factor (50%/100%), Aave V4 computes the exact repayment to restore the borrower to a configurable $HF_{\text{target}} \geq 1.0$:

$$d_{\text{repay}} = \left\lceil \frac{D \cdot u_d \cdot (HF_t - HF)}{(HF_t - \lambda) \cdot p_d^w} \right\rceil \quad (10)$$

D is the debt in units of asset, u_d is the asset unit, p_d^w is the debt asset price, and $\lambda = \beta \cdot CF \cdot 10^{10}$ is the liquidation penalty. This minimizes unnecessary liquidation volume.

5.3 Dutch-Auction Variable Bonus

The liquidation bonus β scales linearly with health factor deterioration. First, the minimum bonus is derived from the maximum via a *liquidation bonus factor* f :

$$\beta_{\min} = \lfloor (\beta_{\max} - 10,000) \cdot f / 10,000 \rfloor + 10,000 \quad (11)$$

If $HF \leq HF_{\text{maxBonus}}$, then $\beta = \beta_{\max}$. Otherwise:

$$\beta = \beta_{\min} + \left\lceil \frac{(\beta_{\max} - \beta_{\min})(HF_{LT} - HF)}{10^{18} - HF_{\text{maxBonus}}} \right\rceil \quad (12)$$

Where HF_{LT} is the target HF liquidation threshold. This creates a time-sensitive liquidation market: as a position deteriorates, the bonus increases, attracting liquidators precisely when intervention is most urgent.

5.4 Dust Prevention and Share-Based Liquidation

A dust threshold of \$1,000 (encoded as $1,000 \times 10^{26}$ in internal value units) prevents residual positions that are uneconomical to liquidate. If the remaining debt or collateral post-liquidation falls below this threshold, the protocol expands the liquidation to fully exhaust

the smaller side, or reverts if the liquidator's intended `debtToCover` is insufficient.

Liquidators may opt to receive *added shares* instead of underlying tokens when `receiveSharesEnabled` is set. This enables capital-efficient liquidation even when Hub liquidity is temporarily low, as no tokens leave the Hub.

5.5 Liquidation Fee

A per-configuration liquidation fee f_l is charged on the bonus portion of seized collateral:

$$S_{\text{fee}} = \left\lceil S_{\text{liq}} \cdot \frac{f_l \cdot (\beta - 10,000)}{\beta \cdot 10,000} \right\rceil \quad (13)$$

where S_{liq} are total collateral shares liquidated and S_{fee} are shares retained by the protocol via `payFeeShares()`.

6 Reinvestment Module

Idle Hub liquidity—assets deposited but not borrowed—can be deployed to external yield strategies through the Reinvestment Module. Two operations are exposed:

1. `sweep(asset, amount)`: transfers underlying tokens from the Hub to the reinvestment controller, reducing `liquidity` and increasing `swept`.
2. `reclaim(asset, amount)`: returns tokens to the Hub, increasing `liquidity` and decreasing `swept`.

Both operations are restricted to the asset's designated `reinvestmentController` address. Setting this address to zero is only permitted when $W = 0$ (all capital reclaimed). Since swept capital remains in the utilization denominator ($U = D / (L + D + W)$), reinvestment is interest-rate-neutral: deploying idle capital does not raise borrowing costs. Any losses from external strategies are absorbed by governance, not depositors.

7 Fee System

The Hub charges a *liquidity fee* f (BPS, per asset) on the growth of aggregate owed amounts:

$$F_u = \left\lceil \frac{(\lceil A_{\text{owed}}(I'_d) / 10^{27} \rceil - \lceil A_{\text{owed}}(I_d) / 10^{27} \rceil) \cdot f}{10,000} \right\rceil \quad (14)$$

where $A_{\text{owed}} = D \cdot I_d + P_{\text{RAY}} + \Delta_{\text{RAY}}$ is the sum of drawn debt, premium debt, and deficit in RAY. Fees apply to *both* drawn and premium interest growth. Unrealized fees are crystallized during each `accrue()` call and later minted as added shares to a designated fee receiver Spoke via `mintFeeShares()`, adjusting supply share holders proportionally.

8 Deficit Elimination

When a liquidation fully exhausts a borrower's collateral but debt remains, the residual debt is recorded as protocol *deficit*:

$$\text{Def} += D_s \cdot I_d + P_{\text{remaining}} \quad (15)$$

where D_s are the borrower’s remaining drawn shares and $P_{\text{remaining}}$ is their remaining premium debt. The borrower’s position is zeroed.

Deficit is eliminated via `eliminateDeficit(asset, amount, coveredSpoke)`. The caller - any active Spoke - requests to absorb up to `amount` of another Spoke’s bad debt, but the protocol only ever eliminates

$$\min(\text{amount}, \delta_{\text{cov}}) \quad (16)$$

where δ_{cov} is the covered Spoke’s outstanding deficit. The deficit therefore need not - and often cannot - be cleared in a single call: a caller may retire it partially, and any request exceeding the remaining deficit is capped at δ_{cov} . Repeated or incremental calls progressively wind the deficit down to zero.

To fund the eliminated portion Def , added shares are burned from the calling Spoke:

$$S_{\text{burn}} = \left\lceil \text{Def} \cdot \frac{S + V}{A + V} \right\rceil \quad (17)$$

Here the symbols are exactly those of the share-accounting model (Section 2.3): A is the total assets currently backing the asset’s added shares, S is the total added shares outstanding for that asset, and $V = 10^6$ is the virtual-offset constant applied to both numerator and denominator. The conversion rounds *up* (the calling Spoke bears the rounding cost, consistent with all share burns). Burning S_{burn} socializes the loss across the covering Spoke’s depositors. This mechanism lets, for example, a DAO treasury Spoke absorb bad debt from any lending Spoke - fully or in increments - without requiring depositor action.

9 Composability Layer

9.1 Tokenization Spoke (ERC-4626)

The `TokenizationSpoke` wraps Hub supply positions as a fully ERC-4626-compliant vault. Vault shares map 1:1 to Hub added shares: `deposit()` calls `hub.add()`, `redeem()` calls `hub.remove()`. Maximum deposit respects the Spoke’s add cap; maximum withdrawal is bounded by available Hub liquidity. The vault supports ERC-2612 `permit()` on both the vault shares and the underlying asset, as well as EIP-712 signature variants (`depositWithSig`, `redeemWithSig`), enabling gasless composability with yield aggregators, DEX routers, and other DeFi primitives.

9.2 Position Managers

To simplify position management, Aave V4 introduces the concept of position managers. Position managers extend Spoke capabilities by building automation mechanisms on top of the base protocol. At launch, three main position managers will be available:

1. **GiverPositionManager**: supply and repay on behalf of users (caller provides funds).

2. **TakerPositionManager**: withdraw and borrow on behalf of users with allowance-based access control and EIP-712 permit signatures.
3. **ConfigPositionManager**: modify collateral settings, risk premium, and dynamic configuration on behalf of users with granular per-capability permissions.

Each position manager must be activated by governance and individually approved by users. All managers support `multicall()` for batching and `rescue()` for recovering stuck tokens.

10 Security and Governance

10.1 Access Control

The Hub uses OpenZeppelin’s `AccessManager` with an enumerable extension for role membership tracking. Hub and Spokes are upgradeable proxies with role-gated administrative functions. All dependencies (OpenZeppelin 5.5.0, Chainlink, Solady, WETH9) are vendored in-tree, eliminating supply chain attack vectors.

10.2 Emergency Controls

A four-tier emergency system provides granular circuit breakers:

- **Hub:** **active/halted** (per-Spoke, per-asset): **active=false** blocks all operations; **halted=true** blocks liquidity-changing operations, but allows premium refresh, deficit reporting and elimination.
- **Spoke:** **paused/frozen** (per-Reserve): **paused** blocks all user operations; **frozen** blocks supply/borrow but permits withdraw, repay, and liquidation.

Transient reentrancy guards (`ReentrancyGuardTransient`) protect all state-mutating Spoke functions. Asset decimals are constrained to [6, 18], and oracle prices are validated as strictly positive.

10.3 Audits

The protocol was subjected to two audit rounds plus a public contest. First round (Sep-Nov 2025): Certora, ChainSecurity, Trail of Bits, and Blackthorn, alongside independent researchers Stermi, Deadrosesecxyz, Josselin, and Kurt Barry. Second round (Jan-Mar 2026): Certora, ChainSecurity, Blackthorn, Stermi, and Josselin. A six-week public security contest ran on Sherlock (Dec 2025 - Jan 2026) with more than 900 verified participants and more than 950 findings submitted, none of which were valid. Formal verification was embedded from the early stages of the codebase, with Certora engaged alongside development rather than only validating finished code. Not all audit reports are published as of this document’s writing; additional reports will be added as they are finalized.

11 Conclusion

Aave V4 represents a structural evolution of decentralized lending. Separating liquidity coordination from borrowing

logic removes the migration friction that made every new strategy expensive, so heterogeneous borrowing strategies deploy against liquidity that already exists. Pricing collateral risk into the borrower's rate ends a cross-subsidy that became increasingly distortional as the range of acceptable collateral widened, and versioned risk configuration lets governance respond to changing conditions without repricing positions opened under different parameters. The liquidation engine restores positions to a target health factor rather than closing a fixed fraction of them, while interest-rate-neutral reinvestment, ERC-4626 tokenization, and EIP-712 meta-transaction support extend the same capital efficiency to the composability layer.

Together, these mechanisms produce a protocol that is more modular, more capital-efficient and more risk-aware, and is suited to the growing diversity of onchain collateral.